Protocol Audit Report

Version 1.0

alva-p.xyz

March 16, 2026

Protocol Audit Report

alva-p

March 16, 2026

Prepared by: alva-p Lead Auditors: - Pineda Álvaro

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

PasswordStore is a Protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not desgined to be used by multiple users. Only the owner should be able to set and acces this password.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: Finding summary

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash:

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

```
./src/
-  PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

I will conduct this audit on March 16, 2026, together with Patrick Collins.

Issues found

Table 2: Issues found summary

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visable to anyone and no longer private.

Description: all data stored onchain is visible to anyone and can be read directly from the blockchain. `PasswordStore::s_password` variable is intended to be a private variable and only accesed

through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract. We show on much method of reading any datta off chain below.

Impact: Anyone can read the private password, severly breaking the functionality of the protocol.

Proof of Concept: (Proof of Code) The below test case shows how anyone can read the password directyle frm the blockchain.

1- Create a locally runing chain `anvil`

2- Deploy the contract on chain

`make deploy`

3- Run the storage tool We use 1 because thats the storage slot of `s_password` in the conract

cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 –rpc-url http://127.0.0.1:8545

output: `0x6d7950617373776f72640000000000000000000000000000000000000000000014`

We can parse that hex to string with:

cast parse-bytes32-string 0x6d7950617373776f72640000000000000000000000000000000000000000000014

output: `myPassword`

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihold & Impact - Impact: HIGH - Likelihood: HIGH - Severity: HIGH

[H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password

Description: The `PasswordStore::setPassword` function is set to be an `external` function, however, the natsepc of the funciont and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
    function setPassword(string memory newPassword) external {
@>      // @audit - there are no access controls
        s_password = newPassword;
        emit SetNetPassword();
    }
```

Impact: Anyone can set/change the password of the contract, severly breaking the contract intended funcionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

```
function test_anyone_can_set_password(address randomAddress) public {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory expectedPassword = "hackedPassword";
        passwordStore.setPassword(expectedPassword);


        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(actualPassword, expectedPassword);
    }
```

Recommended Mitigation: Add an acces control conditional to the `setPassword` function.

```
if(msg.sender != s_owner) {
    revert PasswordStore_NotOwner():
}
```

Likelihold & Impact - Impact: HIGH - Likelihood: HIGH - Severity: HIGH

Informational

[I-#] The `PasswordStore::getPassword` natspec indicates a parameter that doesnt exist, causing the natspec to be incorrect

Description:

```
/*
     * @notice This allows only the owner to retrieve the password.
     * @param newPassword The new password to set.
     // @audit their is no newPassword parameter!
     */
    function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword`function signature is `getPassword()` which the natpsec say it should be `getPassword(string)`

Impact: The natspec is incorrect.

Proof of Concept:

Recommended Mitigation: Remove the incorrect natspect line.

```
-     * @param newPassword The new password to set.
```

Likelihold & Impact - Impact: NONE - Likelihood: - Severity: Information